



unab

**UNIVERSIDAD NACIONAL
GUILLERMO BROWN**

Unidad 1: Mi Primera Tabla: El Ciclo Completo

GESTION DE DATOS (186)

UNIDAD 1: MI PRIMERA TABLA: EL CICLO COMPLETO

OBJETIVOS

- ☐ Analizar un requerimiento de negocio simple para identificar entidades y atributos.
- ☐ Diseñar un modelo conceptual utilizando el Diagrama Entidad-Relación (E-R).
- ☐ Traducir un modelo conceptual a un modelo lógico y físico (CREATE TABLE).
- ☐ Poblar una tabla con datos de prueba (INSERT).
- ☐ Verificar el contenido de una tabla con consultas simples (SELECT).

UNIDAD 1: MI PRIMERA TABLA: EL CICLO COMPLETO

En esta **primera unidad** sentaremos las bases de todo nuestro trabajo. Partiremos de una necesidad de negocio y recorreremos por primera vez el ciclo completo: **desde el análisis** y el **diseño conceptual** en un diagrama, hasta la **construcción, poblado** y consulta de nuestra primera tabla en una base de datos real.

TEMARIO:

En nuestra primera semana, sentaremos las **bases conceptuales** de toda la cursada. Exploraremos la importancia fundamental de las bases de datos en el mundo tecnológico actual y presentaremos el **"mapa" completo del ciclo de diseño** que nos guiará en la construcción de nuestros proyectos. Finalizaremos con nuestro primer ejercicio de análisis de requerimientos.

¿Qué es una base de datos y por qué es necesaria?

Una base de datos es una colección organizada de datos, que están relacionados entre sí y almacenados de forma persistente, es decir, que no se borran al apagar la computadora.



Ciclo de vida de una base de datos



Introducción

En este documento vas a encontrar un recorrido ordenado y explicado paso a paso por los principales conceptos que trabajamos en la **segunda clase** de la materia. A partir de lo que ya construimos juntos en la clase anterior, ahora nos enfocamos en **cómo llevar ese diseño conceptual a una estructura relacional**, es decir, cómo traducir ideas como “entidades” y “atributos” en **tablas, registros y claves reales**.

Mi intención con este material es que puedas **revisar los temas con calma**, hacer conexiones entre los conceptos y entender **por qué organizamos los datos como lo hacemos**. En cada eje vas a encontrar explicaciones, ejemplos, comparaciones y un checklist final con las ideas más importantes para que te sea más fácil estudiar o repasar.

Eje Temático 1: Etapas del diseño de una base de datos relacional

Diseñar una base de datos relacional no es simplemente ponerse a crear tablas. Es un proceso estructurado, que implica entender a fondo qué se quiere representar, cómo se va a organizar la información y de qué manera se va a implementar todo eso en un sistema real. En este eje, quiero que vean con claridad cuáles son las etapas fundamentales de este diseño y por qué es tan importante no saltarse ninguna.

Documentos
idDocumentos INT
Nombre VARCHAR(45)
Autor VARCHAR(45)
Descripción VARCHAR(45)
Tipo VARCHAR(45)
Fecha Creación DATE
idWork Package INT
Proyecto VARCHAR(45)
Indexes

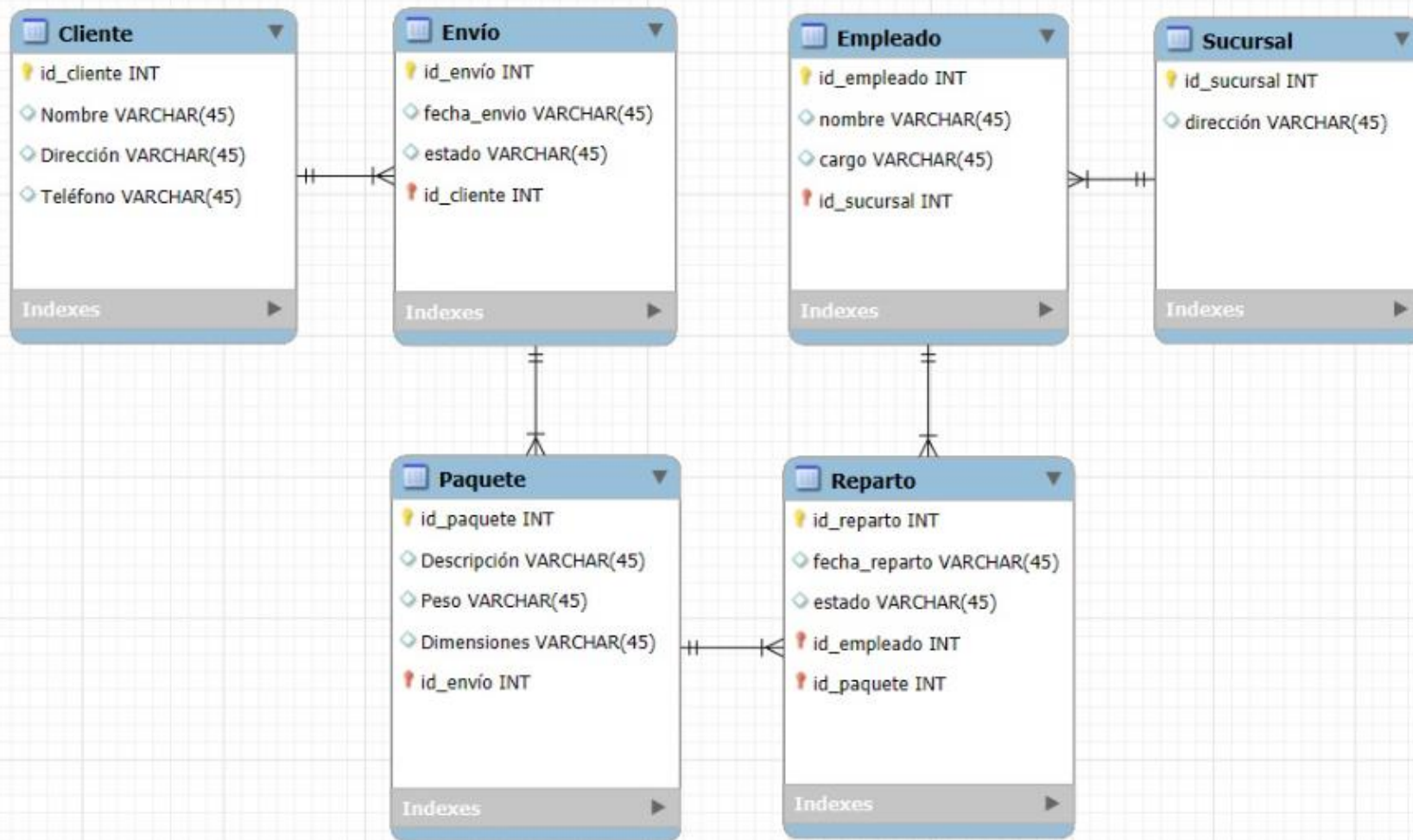
Autores
idAutores INT
Nombre VARCHAR(45)
Apellido VARCHAR(45)
Departamento VARCHAR(45)
Disciplina VARCHAR(45)
Correo VARCHAR(45)
Teléfono VARCHAR(45)
Indexes

Proyectos
idProyectos INT
Nombre Proyecto VARCHAR(45)
Descripción VARCHAR(45)
Fecha Comienzo DATE
Indexes

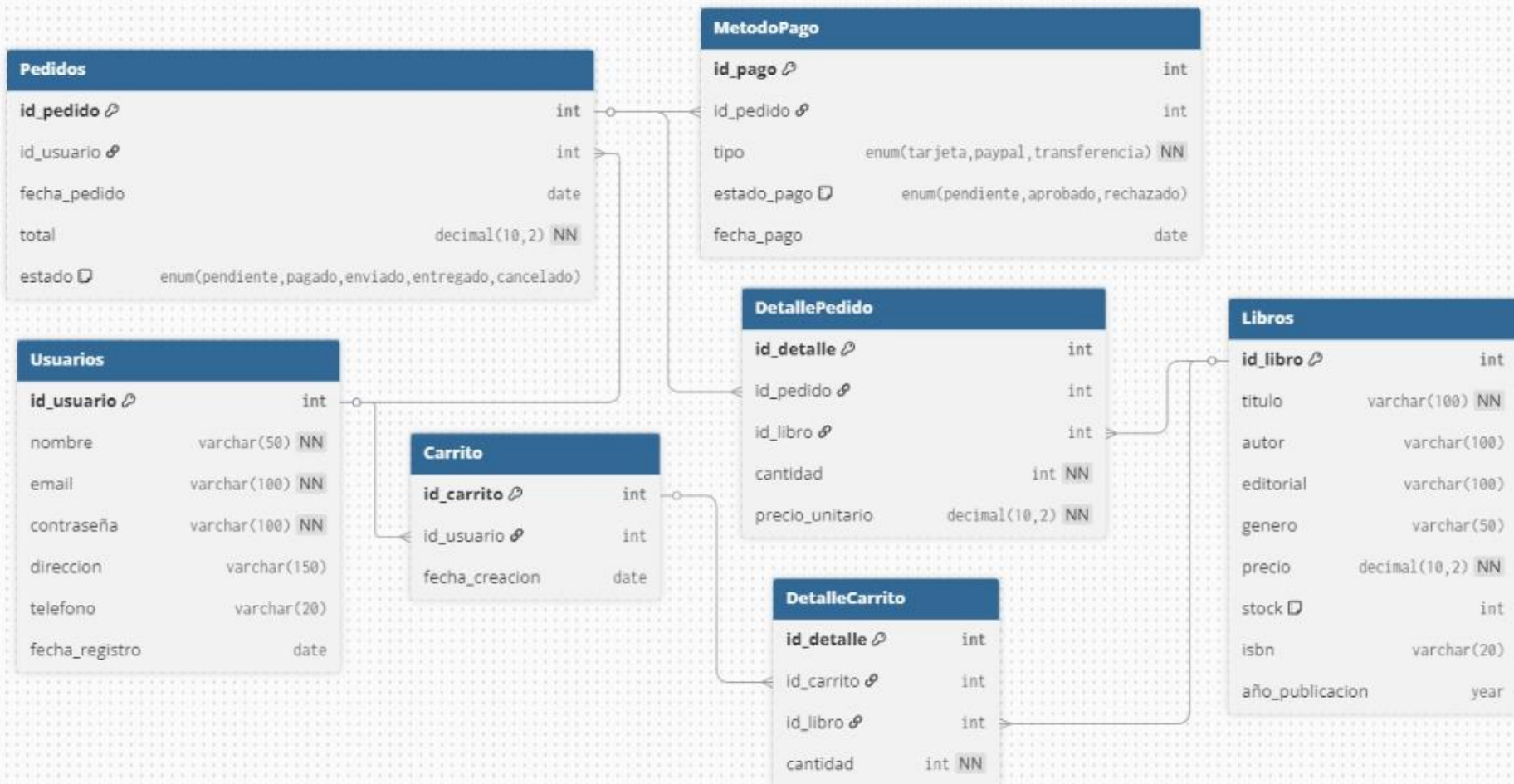
Revisores
idRevisores INT
Nombre VARCHAR(45)
Apellido VARCHAR(45)
Disciplina VARCHAR(45)
Rol VARCHAR(45)
Teléfono LINESTRING
Correo VARCHAR(45)
Indexes

Disciplinas
idDisciplinas INT
Nombre VARCHAR(45)
Descripción VARCHAR(45)
Indexes

Work Package
idWork Package INT
Name VARCHAR(45)
Descripción VARCHAR(45)
Indexes



Socio						Asistencia				
id_socio (PK)	nombre	apellido	dni	telefono	email	fecha_alta	id_asiste	id_socio (F	id_clase	fecha
156	MATIAS	SOSA	36299333	1143324467	msosa@gmail.com	01/02/2022	101	156	1010	12/08/2025
157	NESTOR	GOMEZ	42055423	1163461133	ngomez@hotmail.com	15/03/2023	102	156	1010	14/08/2025
158	NATALIA	MUNA	28336234	1193452345	nati2015@gmail.com	16/03/2023	103	158	1030	01/08/2025
Membresia						Instructor				
id_membresia (PK)	tipo	precio	fecha_inicio	fecha_fin	id_socio	id_instruc	nombre	apellido	especialidad	telefono
1	Mensual	40000	01/01/2025	01/10/2025	156	MM	MARIANA	MONTES	YOGA	113435437
2	Trimestral	35000	01/03/2025	01/12/2025	157	FS	FLORENCIA	SALA	SPINNING	113565665
3	Anual	3000	03/01/2025	01/12/2025	158	TC	TOMAS	CANALE	CROSSFIT	119377222
Clase						Clase_Instructor				
id_clase (PK)	nombre	horario				id_clase	id_instructor (FK → Instructor)			
1010	Crossfit	LMV 20:00				1010	TC			
1020	Spinning	MJS 18:00				1020	FS			
1030	Yoga	LMV 09:00				1030	MM			



Diseño Conceptual: Entender el mundo real

Es la primera etapa, y la más abstracta. En esta etapa, me pregunto: *¿Qué necesito representar del mundo real?* No pienso aún en tablas ni en columnas. Pienso en personas, cosas, situaciones que existen y que quiero representar digitalmente.

¿Qué hago en esta etapa?

- Identifico **entidades**: cosas o personas que tienen existencia propia. Ejemplo: *estudiante, producto, factura, materia*.
- Identifico **atributos**: características que comparten las instancias de una entidad. Ejemplo: todos los estudiantes tienen *nombre, apellido, DNI*.
- Identifico **relaciones** o **interrelaciones**: vínculos entre entidades. Ejemplo: *un estudiante cursa una comisión de una materia*.

 Noten que en este punto no hablamos de claves primarias ni de tablas. Estamos haciendo una especie de "boceto" conceptual.

 Equivalencias terminológicas y representaciones por etapa

Concepto real	Diseño Conceptual	Diseño Lógico (Modelo Relacional)	Diseño Físico (Implementación en SGBD)	Representación
Cosa o persona del mundo real	Entidad	Tabla (también llamada “relación” en teoría relacional)	Tabla implementada en MySQL	◆ Rectángulo en Diagrama E-R / CREATE TABLE
Propiedad común a las instancias	Atributo	Columna o Campo	Columna con tipo de dato definido	◆ Elipse en Diagrama E-R / nombre de columna en SQL
Ejemplar concreto de una entidad	Instancia	Registro, Tupla o Fila	Registro almacenado	📄 Fila en tabla / resultado horizontal en una consulta
Vínculo entre entidades	Relación o Interrelación	Clave foránea que conecta tablas	FOREIGN KEY implementada en SQL	🔗 Rombos en Diagrama E-R / líneas entre claves
Identificador único	(No se explicita aún)	Clave primaria (PK)	PRIMARY KEY en SQL	🔑 Subrayado en Diagrama E-R / definición PRIMARY KEY
Conjunto de estructuras y vínculos	Modelo Entidad-Relación	Modelo Relacional	Base implementada en SGBD	🕒 Diagrama E-R / Esquema / SQL ejecutado

 Entidades e instancias en contexto

Si tengo la entidad Estudiantes, con atributos como *nombre*, *DNI*, *fecha de nacimiento*, entonces cada alumno de esta materia es una **instancia** de esa entidad. Y cada uno tiene valores específicos para cada atributo.

Entonces:

- **Entidad:** Estudiantes
- **Atributos:** nombre, apellido, legajo, DNI, fecha de nacimiento, etc.
- **Instancia:** "Lucas, Martínez, 45678, 39222111, 2003-06-08"

 Relación entre conceptos

Concepto	¿Qué es?	Ejemplo concreto
Entidad	Lo que quiero representar del mundo real	Estudiantes
Atributo	Característica compartida por todas las instancias	Nombre, Apellido, DNI
Instancia	Ejemplar concreto de una entidad	"Lucía", "Ruiz", 40222333

¿Qué es una clave primaria?

Cada vez que armamos una tabla, necesitamos tener algún campo que **distinga a cada fila del resto**. Es decir, un valor que no se repita y que permita identificar de forma clara a cada instancia. Ese campo es la **clave primaria**.

 Una **clave primaria** es un campo (también llamado atributo, característica o columna) que **identifica de forma única cada registro en una tabla**. Es el valor que le dice a la base de datos: *esto es una fila distinta a todas las demás*.

- En la entidad *Estudiante*, ese campo puede ser el *legajo*.
- En la entidad *Producto*, puede ser el *código de barras*.
- En la entidad *Comisión*, puede ser un *código de comisión*.

¿Qué es una clave foránea?

Cuando queremos vincular una tabla con otra, necesitamos un campo que **haga referencia a la clave primaria de esa otra tabla**. Ese campo es la **clave foránea**.

 Una **clave foránea** es un campo que se usa para **establecer una relación entre dos tablas**. Su valor debe coincidir con el de la clave primaria de otra tabla.

Por ejemplo:

- La tabla *Comisión* puede tener un campo que diga a qué *Materia* pertenece. Ese campo será una **clave foránea**.
- Si tenemos una tabla que relaciona estudiantes con comisiones, esa tabla va a tener dos claves foráneas: una que apunta a *Estudiante* y otra a *Comisión*.

 La clave primaria le dice a la tabla: “yo soy esta fila”.

 La clave foránea le dice a otra tabla: “yo estoy relacionado con esa otra fila”.

 Clave primaria vs. clave foránea

Concepto	¿Para qué sirve?	¿Dónde está?	¿Debe ser única?
Clave primaria	Identifica de forma única cada fila	En la misma tabla	✓ Sí
Clave foránea	Relaciona con una fila de otra tabla	En una tabla diferente	✗ No necesariamente

 ¿Por qué son tan importantes?

- Porque garantizan la **integridad de los datos**.
- Porque permiten **relacionar tablas** de forma confiable.
- Porque son la base para hacer **consultas complejas** y unir información.
- Porque sin ellas, no podríamos controlar **errores ni inconsistencias**.

 Más adelante, cuando trabajemos con SQL, vamos a ver cómo se declaran estas claves y qué restricciones podemos aplicar (ON DELETE, ON UPDATE, etc.).

 ¿Cómo se representan en cada etapa del diseño?

Etapas del diseño	Clave primaria	Clave foránea
Diseño Conceptual	Se subraya el atributo clave	No se marca aún
Modelo Relacional	Se indica como PK	Se indica como FK apuntando a otra PK
Diseño Físico (SQL)	PRIMARY KEY (...)	FOREIGN KEY (...) REFERENCES ...

Eje Temático 5: Tablas, registros, columnas y valores

Correspondencias entre el mundo conceptual y el modelo relacional

Hasta acá veníamos hablando de **entidades**, **atributos** y **relaciones**. Pero llegó el momento de conectar todo eso con el **modelo relacional**, que es el formato con el que vamos a trabajar en la práctica.

¿Y qué usamos en ese modelo? **Tablas**.

¿Y qué hay en esas tablas? **Filas y columnas**.

¿Y qué significa cada cosa? Eso es lo que vamos a ver ahora.

Del modelo conceptual al modelo relacional

En el diseño conceptual hablábamos con palabras. Decíamos cosas como “una materia tiene nombre, código y carga horaria”.

Pero en el modelo relacional todo eso se traduce a una estructura tabular: **una tabla con columnas y filas**.

Diseño Conceptual	Modelo Relacional
Entidad	Tabla
Atributo	Columna (campo/característica)
Instancia	Fila (registro/tupla)
Valor del atributo	Dato en una celda

 Lo importante no es solo memorizar estos nombres, sino **entender cómo se corresponden**.

¿Qué es la redundancia?

La **redundancia** ocurre cuando **el mismo dato aparece muchas veces en distintos lugares**. Y eso, en bases de datos, puede traernos varios problemas.

 La redundancia es tener datos repetidos sin necesidad. No solo ocupa espacio, sino que genera **inconsistencias**: si cambio algo en un lugar pero me olvido de cambiarlo en otro, los datos dejan de coincidir.

Ejemplo clásico:

legajo	nombre	materia	docente
1032	Laura	Bases de Datos	Daniel Miniello
1032	Laura	Inglés I	Daniel Miniello

¿Ves que el nombre de Laura aparece dos veces? ¿Y el de los docentes también? Esa estructura parece práctica al principio, pero **no escala bien**. ¿Qué pasa si cambia el nombre de un docente? ¿Tengo que ir a todas las filas donde aparece?

¿Por qué evitarla?

Porque:

- Aumenta el **riesgo de errores**.
- Hace que sea más difícil **modificar datos**.
- Ocupa **espacio innecesario**.
- Complica las **consultas** y el **mantenimiento**.

¿Cómo se soluciona?

Dividiendo la información en **varias tablas**, cada una centrada en una entidad específica, y relacionándolas correctamente con claves.

En el ejemplo anterior, podríamos tener:

1. Una tabla Estudiantes
2. Una tabla Materias
3. Una tabla Docentes
4. Una tabla intermedia Inscripciones, que relacione todo

Eso evita repetir los datos. El nombre de Laura estará **una sola vez**, igual que el nombre del docente y el nombre de la materia.

 La solución no es “no repetir nunca”, sino **no repetir lo que no hace falta repetir**.

Lo que estamos anticipando: la normalización

Aunque no usamos todavía ese término, lo que estamos haciendo es **anticipar el criterio de normalización**: organizar los datos en tablas separadas, con claves claras, para **evitar redundancia y asegurar consistencia**.

Clasificación de los Atributos

1. Atributos Simples (o Atómicos)

- Son aquellos que **no pueden dividirse** en otras partes que tengan significado independiente dentro del modelo.
- Es decir, no tienen **componentes significativos**: sus partes no serían útiles como atributos por separado.

Ejemplos:

- edad: no tiene subpartes relevantes.
- DNI: es una unidad completa, no se divide en componentes con sentido propio.
- precio: tampoco tiene subdivisiones útiles en este contexto.

 En MySQL se implementan como columnas con tipos de datos básicos como `INT, VARCHAR, DATE`, etc.

2. Atributos Compuestos

- Son atributos que **sí pueden dividirse** en otros atributos más simples, y cada uno de esos componentes tiene significado propio y puede analizarse por separado.

Ejemplos:

- nombre_completo → puede dividirse en nombre y apellido
- dirección → puede dividirse en calle, número, ciudad, provincia, código postal

 En el diseño físico (MySQL), suele ser conveniente descomponer estos atributos para permitir búsquedas, ordenamientos y evitar ambigüedades.

3. Atributos Multivaluados

- Son atributos que **pueden contener más de un valor** para una misma instancia de entidad.
- Esto quiere decir que **una sola entidad puede estar asociada a varios valores del mismo atributo**.

◆ Ejemplos:

- Una persona puede tener **más de un número de teléfono**.
- Un estudiante puede tener **varios correos electrónicos**.

📌 En el modelo relacional, no se representan directamente como columnas. Se resuelven mediante **una tabla adicional** que permita vincular múltiples valores al mismo registro de la entidad principal.

5. Atributos Almacenados

- Son atributos cuyos **valores se guardan físicamente** en la base de datos.
- No se calculan ni derivan de otros datos: deben ser **ingresados o capturados directamente**.

◆ Ejemplos:

- fecha de nacimiento
- precio_unitario
- cantidad

 Estos atributos existen físicamente en las tablas y forman parte del almacenamiento de la base de datos.

6. Atributos Derivados

- Son atributos cuyos valores **se obtienen a partir de otros atributos**.
- No es necesario almacenarlos, porque se pueden **calcular cuando se los necesita**.

◆ Ejemplos:

- edad puede calcularse a partir de fecha de nacimiento
- importe_total puede derivarse de cantidad × precio_unitario

 En MySQL pueden implementarse como:

- **Campos calculados** en consultas (SELECT)
- **Vistas**
- **Triggers**, si decidimos almacenarlos pero queremos mantenerlos actualizados automáticamente

¿Qué es un Sistema de Gestión de Base de Datos (SGBD)?

Una base de datos por sí sola es solo un conjunto de datos almacenados. Para poder crear, consultar, modificar o eliminar esos datos de forma eficiente y segura, se necesita una herramienta específica: el Sistema de Gestión de Base de Datos, o SGBD (en inglés: DBMS – DataBase Management System).

¿Qué hace un SGBD?

Un SGBD es un software especializado que permite:

- ☐ Crear y estructurar una base de datos.
- ☐ Insertar nuevos datos.
- ☐ Consultar datos existentes.
- ☐ Modificar datos.
- ☐ Eliminar datos.

- ☐ Controlar el acceso y la seguridad de los datos
- ☐ Asegurar la integridad y coherencia de los datos

Algunos ejemplos conocidos de SGBD

Nombre	Características principales
MySQL	Libre, muy popular, ideal para sitios web y aplicaciones.
PostgreSQL	Libre, muy robusto, con muchas funciones avanzadas.
SQL Server	De Microsoft. Potente, orientado a empresas.
Oracle Database	Muy potente, pago. Usado en grandes empresas.
MariaDB	Derivado de MySQL, también libre y muy utilizado.
SQLite	Ligero, sin necesidad de instalación, usado en <u>apps</u> móviles.

¿Y qué lenguaje usan los SGBD?

La mayoría de los SGBD trabajan con un lenguaje llamado SQL (Structured Query Language), que sirve para consultar y manipular los datos.

En esta materia vamos a trabajar con MySQL y MySQL Workbench, que permiten usar SQL en un entorno visual y práctico.

Un SGBD no es lo mismo que una base de datos

Consultas
